

16

CNN

Convolutional Neural Networks — built for images

How computers learn to SEE

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A neural network designed for images
Full name	Convolutional Neural Network
Key idea	Slide small filters over an image to find patterns
Powers	Image classification, object detection, face recognition
You've used it	ResNet18, EfficientNet — both are CNNs

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	Why a Special Network for Images?	The problem with normal nets
2	An Image is Just Numbers	How computers see pictures
3	Convolution	The filter that slides and finds patterns
4	Filters and Feature Maps	What the filters detect
5	Pooling	Shrinking while keeping what matters
6	The Full CNN	Putting all the layers together
7	Building a CNN in Code	A real PyTorch example
8	Famous CNNs	LeNet to EfficientNet
9	Transfer Learning	Standing on giants' shoulders
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. Why a Special Network for Images?

A normal neural network connects every input to every neuron. For a small image this is okay. But a real photo has millions of pixels. Connecting every pixel to every neuron would need billions of weights — far too many to train, and it would overfit instantly.

There's a deeper problem too. A normal network treats each pixel separately, with no idea that nearby pixels belong together. But in images, **position matters** — an eye is a group of pixels in a certain pattern, wherever it appears in the photo.

CNNs (Convolutional Neural Networks — ) solve both problems. They look at small patches of the image at a time, reuse the same pattern-detectors across the whole image, and build up understanding from simple edges to whole objects.

■ *This is the network behind your Product Image Classifier (ResNet18) and Visual Defect Detector (EfficientNet). This topic explains what's happening inside those models.*

2. An Image is Just Numbers

To a computer, an image is a grid of numbers. Each **pixel** () is a number saying how bright it is. For a black-and-white image, 0 is black and 255 is white, with greys in between.

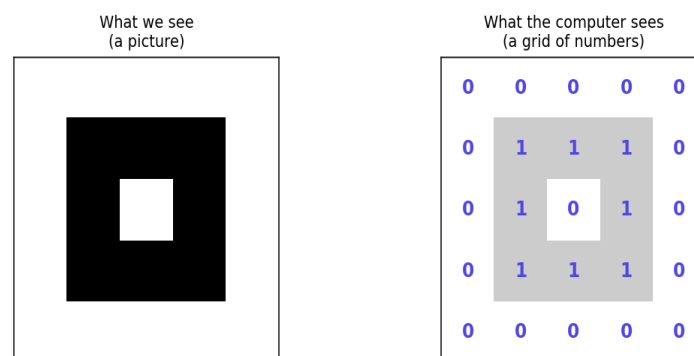


Fig 1 — We see a picture; the computer sees a grid of numbers. CNNs work on these numbers.

A colour image has three of these grids stacked — one for Red, one for Green, one for Blue (RGB). So a colour photo is a 3-D tensor: height × width × 3 colours. That's the input a CNN works on.

3. Convolution — The Core Idea

Convolution (■■■■■■■■■■) is the heart of a CNN. The idea: take a tiny grid of numbers called a **filter** (■■■■■■■■■■), and slide it across the whole image. At each spot, it checks 'does this little patch match my pattern?' and writes down the answer.

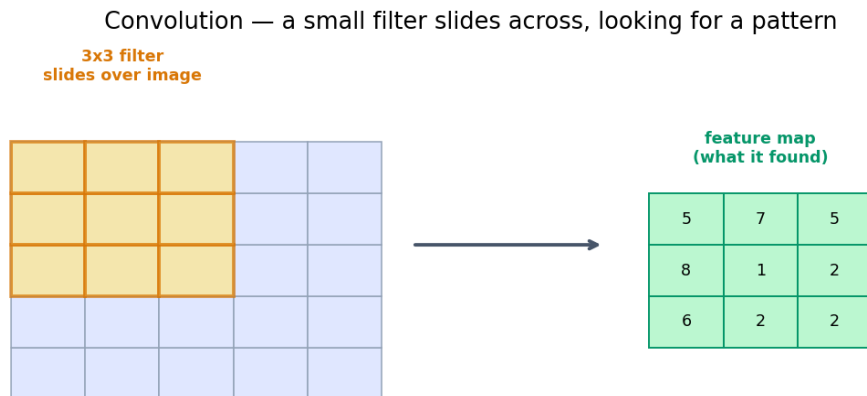


Fig 2 — A small filter slides across the image. At each position it produces one number, building a feature map.

Think of the filter as a tiny stamp looking for one specific thing — maybe a vertical edge. As it slides over the image, wherever it finds a vertical edge it lights up; elsewhere it stays quiet. The result is a new grid showing WHERE that pattern was found, called a **feature map** (■■■■■■ ■■■■■■).

Two key benefits of this sliding approach:

- **Same filter everywhere** — one edge-detector works across the whole image, so you need far fewer weights than a normal network.
- **Position-independent** — it finds a cat's ear whether it's top-left or bottom-right, because the same filter slides everywhere.

4. Filters and Feature Maps

A CNN doesn't use just one filter — it uses many, and it **learns** them during training. You don't tell it what to look for; it figures out the useful patterns by itself.

The beautiful part is how patterns build up across layers, from simple to complex:

Layer depth	What its filters detect
Early layers	Simple things: edges, corners, colours
Middle layers	Combinations: textures, shapes, eyes, wheels
Deep layers	Whole objects: faces, cats, cars, defects

So the first layers find tiny pieces, the next layers combine those pieces into parts, and the deepest layers combine parts into whole objects. The network builds understanding step by step — exactly how it learns to recognise a defect on a product or tell a shirt from shoes.

5. Pooling — Shrinking Smartly

After convolution, the feature maps can be large. **Pooling** (■■■■■) shrinks them while keeping the important information. The most common type is **max pooling**: look at a small block and keep only the biggest number.

Max Pooling — shrink by keeping the biggest value in each block

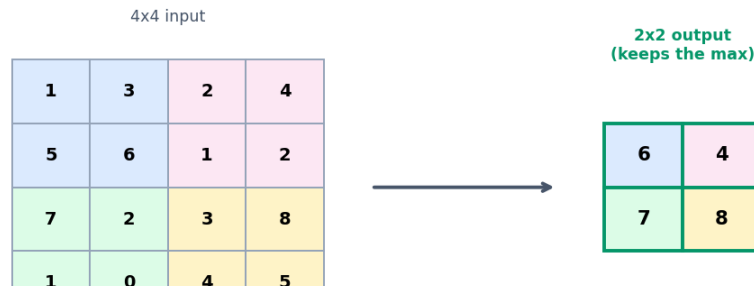


Fig 3 — Max pooling looks at each 2x2 block and keeps only the largest value, shrinking the grid.

Why pooling helps:

- **Smaller data** — fewer numbers to process, so the network is faster.
- **Keeps the strongest signals** — the max value is the strongest 'I found it!' response.
- **More robust** — small shifts in the image don't change the result much.

■ A CNN usually alternates: convolution to find features, then pooling to shrink, then more convolution, more pooling. Each round, the image gets smaller but richer in meaning.

6. The Full CNN — Putting It Together

A complete CNN has two parts. First, the **feature-finding** part (convolution + pooling, repeated). Second, the **decision-making** part (normal dense layers, like the networks from earlier topics) that takes the found features and makes the final choice.

A Full CNN: feature-finding layers, then decision layers

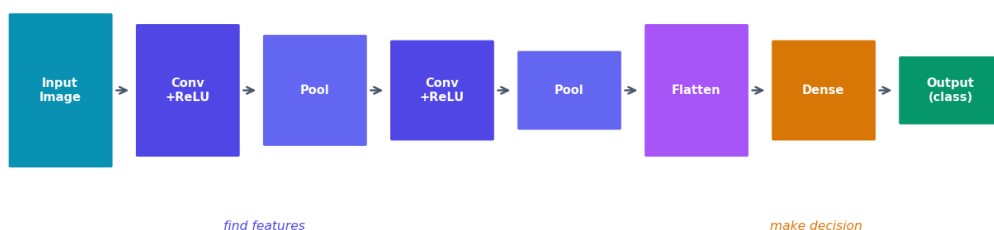


Fig 4 — A full CNN: convolution and pooling find features, then flatten and dense layers decide the class.

Part	Job
Conv + ReLU	Find patterns (edges, shapes, objects)
Pooling	Shrink and keep the strongest signals

Flatten	Turn the 2-D feature maps into a flat list of numbers
Dense layers	Use the features to make the final decision
Output (softmax)	Give the probability of each class

7. Building a CNN in Code

Here's a small CNN in PyTorch for classifying images into 10 classes:

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Feature-finding part
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()
        # Decision part
        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10) # 10 classes

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x))) # conv -> pool
        x = self.pool(self.relu(self.conv2(x))) # conv -> pool
        x = x.flatten(1) # flatten to a list
        x = self.relu(self.fc1(x)) # dense
        x = self.fc2(x) # output
        return x

model = SimpleCNN()
```

■ *Conv2d is the convolution layer. The numbers (3, 16) mean '3 colour channels in, 16 filters out'. MaxPool2d(2,2) halves the height and width. This is exactly the structure of the diagram above.*

8. Famous CNN Architectures

Over the years, researchers designed famous CNN architectures, each better than the last. You'll see these names everywhere:

Name	Claim to fame
LeNet (1998)	The first successful CNN — read handwritten digits
AlexNet (2012)	Won ImageNet, started the deep learning boom
VGG (2014)	Simple, deep, many small filters
ResNet (2015)	Added 'skip connections' to train very deep nets
EfficientNet (2019)	Best accuracy for its size — smart scaling

ResNet (which you used as ResNet18) introduced skip connections — shortcuts that let information jump over layers. This solved a big problem and made it possible to train networks hundreds of layers deep. **EfficientNet** (your defect detector) is famous for getting top accuracy while staying small and fast, by

scaling depth, width, and image size together.

9. Transfer Learning — Don't Start From Zero

Training a big CNN from scratch needs millions of images and huge computing power. **Transfer learning** (■■■■■■■■■■ ■■■■■■■■) is the smart shortcut: take a CNN that's already been trained on millions of images, and reuse it for your own smaller task.

The pre-trained network already knows how to detect edges, shapes, and textures — that knowledge transfers to almost any image task. You just replace the final decision layer with your own, and train only that part (or fine-tune the whole thing gently). This is how you got 99.3% accuracy on your image classifier without millions of images.

```
import torch.nn as nn
from torchvision import models

# Load a CNN already trained on millions of images
model = models.resnet18(weights='DEFAULT')

# Freeze the feature-finding layers (keep their knowledge)
for param in model.parameters():
    param.requires_grad = False

# Replace the final layer for YOUR number of classes
model.fc = nn.Linear(model.fc.in_features, 5)  # 5 classes

# Now train only the new final layer on your data
```

■ *Transfer learning is the standard approach for almost all real image projects. Starting from a pre-trained model saves time, needs far less data, and usually gives better results. You used this in both your image projects.*

10. Common Mistakes

Mistake	Why it's a problem	Fix
Not normalising images	Network trains poorly	Scale pixels (e.g. 0-1) first
Wrong input size	Shape mismatch crash	Resize images to what the model expects
Training from scratch	Needs huge data + time	Use transfer learning
Too small a dataset	Overfits badly	Use data augmentation + pretrained model
Forgetting to flatten	Shape error before dense layers	flatten() after the conv part
No data augmentation	Misses easy accuracy gains	Flip / rotate / crop training images

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
----------------	-----------------

Image to computer	a grid of pixel numbers (0-255)
Convolution layer	<code>nn.Conv2d(in_ch, out_ch, kernel_size=3)</code>
Activation	<code>nn.ReLU()</code>
Pooling layer	<code>nn.MaxPool2d(2, 2)</code>
Flatten	<code>x.flatten(1)</code>
Dense layer	<code>nn.Linear(in, out)</code>
Filter	small grid that detects one pattern
Feature map	output showing where a pattern was found
Pooling	shrink by keeping the max value
Load pretrained	<code>models.resnet18(weights='DEFAULT')</code>
Freeze layers	<code>param.requires_grad = False</code>
Replace final layer	<code>model.fc = nn.Linear(in, n_classes)</code>
Data augmentation	<code>transforms.RandomHorizontalFlip()</code>

The one idea to remember: a CNN slides small filters over an image to find patterns, pools to shrink, and repeats — building from edges to shapes to whole objects. Then dense layers make the final decision.

Next Topic: LSTM (Long Short-Term Memory) — a network built for sequences like text and time. This is the engine behind your Twitter sentiment analysis project.